

Java SDE Master Plan

Java SDE — Master Plan (No Mixing, One Track Only)

Rule #0: You are only ever allowed to be “in” ONE stage at a time. You do not touch DSA while still in Java Basics. You do not touch Spring Boot while still in DSA. Each stage has a **LOCK**. You cannot unlock the next stage until you pass the checkpoint of the current one.

0. VS Code Setup (Do This First — Once)

1. Install **JDK 17 or 21** (not JRE — the full JDK).
2. Open VS Code → Extensions → install “**Extension Pack for Java**” (by Microsoft). This alone gives you: syntax highlighting, run/debug, IntelliSense, Maven support.
3. Create a folder `java-practice/`. Open it in VS Code.
4. Create `Main.java`, paste:

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello Java");
    }
}
```

5. Press **Run** (top-right ► button that appears above `main`). If output prints — setup is done. Move on. Do not spend more than 30 minutes on setup.
-

1. THE DAILY LOOP (Use This Every Single Day, No Exceptions)

Same 4 steps, every day, regardless of stage:

Step	Time	Action
1. Learn	25-30 min	ONE concept only. Not two. Not “a bit of the next one too.”
2. Write	30-40 min	Type 3-5 of your own examples. No copy-paste.
3. Practice	20-30 min	Solve 3-5 small problems on that concept.
4. Review	10 min	Write 3-5 lines in a notes file: what you learned + 1 mistake you made.

If you only have 1 hour: 25 min learn + 25 min code + 10 min review. Never skip a day entirely — do a 15-minute version instead.

Keep one file: `notes.md`. One entry per day. That's your only tracking tool. No apps, no trackers, no dashboards.

2. THE ONE TRACK (Sequential Stages — Do Not Reorder)

STAGE 1	Java Basics	(Weeks 1–4)
	↓ (locked until checkpoint passed)	
STAGE 2	OOP + Core Java	(Weeks 5–8)
	↓	
STAGE 3	Collections + Project	(Weeks 9–12)
	↓	
STAGE 4	DSA	(Months 4–8)
	↓	
STAGE 5	SQL + DBMS	(Month 9)
	↓	
STAGE 6	Git/GitHub	(Month 10)
	↓	
STAGE 7	Spring Boot	(Months 11–15)
	↓	
STAGE 8	2–3 Real Projects	(Months 16–18)
	↓	
STAGE 9	Docker + Linux + Cloud + Testing	(Months 19–21)
	↓	
STAGE 10	System Design + Interviews	(Months 22–24)

You are allowed to peek at what's ahead for motivation. You are **not** allowed to study ahead of your current stage.

3. STAGE 1 — Java Basics (Weeks 1-4)

Week 1 — Syntax

- Day 1: class + main method + Hello World (setup above).
- Day 2: variables — `int`, `double`, `char`, `boolean`, `String`. Write 5 programs.
- Day 3: `Scanner` input + `System.out.println` output. Write 5 programs.
- Day 4: arithmetic, comparison, logical operators. Write 5 programs.
- Day 5-6: practice — print name, add two numbers, area of shapes, temperature converter, simple interest.
- Day 7: review everything from Week 1, no new topic.

Checkpoint before Week 2: Can you write a program from a blank file, with no notes open, that takes 2 numbers as input and prints their sum, product, and average? If yes → continue. If no → repeat Week 1 for 2-3 more days.

Week 2 — Conditions + Loops

- Day 1: `if / else / else-if / switch`.
- Day 2: `for` loop.
- Day 3: `while` and `do-while`.
- Day 4: nested loops + `break / continue`.
- Day 5-6: practice — even/odd, largest of 3 numbers, grade calculator, multiplication table, factorial, prime check, palindrome number, Fibonacci.
- Day 7: review.

Checkpoint: Write a prime-number checker and a multiplication table generator from scratch, no notes.

Week 3 — Methods + Arrays + Strings

- Day 1: methods, return values, parameters.
- Day 2: 1D arrays — declare, loop, print.
- Day 3: 2D arrays.
- Day 4: String methods (`length`, `charAt`, `substring`, `equals`, `split`).
- Day 5: linear search + basic array problems.
- Day 6-7: practice — max/min in array, reverse array, count frequency, search element, reverse a string, palindrome string, count vowels.

Checkpoint: Write a method that takes an array and returns the max value — no notes.

Week 4 — First OOP

- Day 1: class + object.
- Day 2: fields + methods.
- Day 3: constructors + `this` .
- Day 4: access modifiers + basic encapsulation.
- Day 5-7: **Build:** Student Management System (add student, display all, search by name) using arrays or `ArrayList` .

STAGE 1 FINAL CHECKPOINT (must pass to unlock Stage 2): - [] Can explain class vs object without notes. - [] Can write a loop + array program cold. - [] Finished the Student Management System yourself (80%+ own code).

If any box is unchecked, do not move to Stage 2. Spend extra days here — this is normal and expected.

4. STAGE 2 — OOP + Core Java (Weeks 5-8)

- Inheritance, `super` , method overriding.
- Polymorphism (compile-time + runtime).
- Abstract classes vs interfaces.
- Exception handling (`try/catch/finally` , custom exceptions).
- `ArrayList` , `HashMap` , `HashSet` (just usage, not internals yet).
- **Build:** extend the Student Management System — add inheritance (e.g. `Student` → `GraduateStudent`), add exception handling for invalid input.

Checkpoint to unlock Stage 3: Explain interface vs abstract class in your own words + show it working in code.

5. STAGE 3 — Collections + Small Project (Weeks 9-12)

- `List` , `Set` , `Map` in depth — when to use which.
- `Iterator` , sorting with `Comparator` .
- File I/O basics (read/write a text file).
- **Build:** a small complete project — e.g. Library Management System or Contact Book, using collections + file storage (no database yet).

Checkpoint to unlock DSA: You should now be comfortable with variables, loops, methods, arrays, strings, OOP, and collections. You do NOT need to have mastered everything — “comfortable,” not “expert.”

6. STAGE 4 — DSA (Months 4-8)

Order — do not skip ahead in this list: 1. Big-O basics 2. Arrays 3. Strings 4. Two pointers 5. Sliding window 6. Prefix sum 7. Searching / Binary search 8. Sorting 9. Recursion 10. Linked list 11. Stack and Queue 12. HashMap and HashSet 13. Trees and BST 14. Heap / PriorityQueue 15. Graphs 16. Greedy 17. Backtracking 18. Dynamic Programming

Weekly pattern per topic (repeat for every topic above):

Day	Task
Mon	Learn concept + 2 easy problems
Tue	3 easy problems
Wed	2 easy + 1 medium
Thu	3 problems, no notes allowed
Fri	Review mistakes, re-solve them
Sat	Mixed practice (this + previous topics)
Sun	Rest / light revision only

For every problem, write this exact structure in your notes:

```
Pattern:
Approach:
Complexity:
Code:
Mistake I made:
```

Always attempt alone first. Only after a genuine attempt, look at a hint. Then close it and re-write the solution from memory — never copy while looking.

Checkpoint to unlock SQL: You’ve been through all 18 topics at least once, and can solve a “medium” problem in a familiar pattern (e.g. arrays, two pointers) within 25-30 minutes without help.

7. STAGE 5 — SQL + DBMS (Month 9)

- SELECT, WHERE, ORDER BY
 - GROUP BY, HAVING, aggregate functions
 - INSERT, UPDATE, DELETE
 - INNER JOIN, LEFT JOIN
 - Primary/foreign keys, indexes
 - Normalization
 - Transactions and ACID
 - **Build:** one small database (e.g. for the Library project), then connect it to Java using JDBC.
-

8. STAGE 6 — Git/GitHub (Month 10)

- init, add, commit, push, pull, clone
- Branching + merging
- Resolving a merge conflict (practice this deliberately)
- Push all previous projects to GitHub with proper README files.

9. STAGE 7 — Spring Boot (Months 11-15)

Flow to understand first: Client → Controller → Service → Repository → Database - Dependency injection, project structure - REST controllers - Services + repositories - DTOs + validation - JPA/Hibernate + relationships - Exception handling - Spring Security + JWT - JUnit + Mockito

10. STAGE 8 — Real Projects (Months 16-18)

Rule: **20% tutorial reference, 80% your own code.** Progression: 1. Beginner (already done in earlier stages): calculator, student system, library system. 2. Intermediate: expense tracker or employee management. 3. Backend: e-commerce API — the big one, built in phases:

Week	Focus
1	Requirements + DB design
2	User registration
3	Login/security
4	Products/categories
5	Cart
6	Orders
7	Validation + error handling
8	Tests + docs + deployment

4. Advanced: job portal or your own original idea.

You need **2-3 strong projects you can explain line-by-line** — not 15 shallow ones.

11. STAGE 9 — Docker + Linux + Cloud + Testing (Months 19-21)

- Linux basics (navigation, permissions, processes)
 - Docker (containerize your Spring Boot app)
 - Deploy to a cloud service (e.g. Render, Railway, AWS free tier)
 - Proper testing (unit + integration) on your main project
-

12. STAGE 10 — System Design + Interview Prep (Months 22-24)

- System design fundamentals (scalability, caching, load balancing, DB design at scale)
 - Revisit DSA patterns intensively
 - Mock interviews
 - Resume + portfolio polish
-

12.5 How to Handle Things “Side by Side” (Primary vs Maintenance)

This is the #1 confusion point, so here's the exact rule.

There are two categories of things in this plan — never treat them the same:

Category	What it means	Examples
SUBJECT	Needs your full, undivided focus while you're learning it. Only ONE subject is "Primary" at a time.	Java Basics, OOP, DSA (first pass), SQL, Spring Boot
TOOL	Something you <i>use</i> daily in the background while building things — not something you sit down and "study" separately.	Git (day-to-day commits), your notes file

Direct answers to your question:

- **"Do Java basics then DSA, or DSA with it?"** → Java Basics (Stages 1-3) **FIRST**, fully, alone. Do not touch DSA during this time. DSA needs arrays/loops/methods to already be comfortable, or every DSA problem becomes two problems (learning syntax + learning the pattern) at once — that's what burns people out.
- **"Do GitHub with it?"** → Yes, always, from Day 1 — but only as a *tool*, not a subject. Every project you build (even Week 1 practice files), run `git init`, `git add .`, `git commit -m "..."`. That's it. Don't study branching/merging/rebasing yet — that formal study happens in Stage 6 (Month 10). Using Git ≠ studying Git.
- **"Do DSA with Spring Boot / SQL / Projects?"** → Only **AFTER** your first full pass through all 18 DSA patterns (end of Stage 4). From Stage 5 onward, DSA switches from Primary to **Maintenance**: 20-30 min/day, 3 problems, just to keep patterns sharp — while your Primary focus moves to SQL, then Spring Boot, then Projects. DSA maintenance never fully stops again until you're placed — interview prep is continuous, not a phase you finish once.

Full Primary vs Maintenance map:

Stage	Primary (majority of your time)	Maintenance (small, parallel, background)
1-3 Java Basics + OOP + Collections	Java fundamentals	Git used casually (commit your practice files) — not studied
4 DSA — first pass	All 18 patterns	Nothing else — full focus, this is the one stage with zero parallel subject
5 SQL	SQL + DBMS	DSA revision: 20-30 min/day, 3 problems
6 Git/GitHub (formal)	Branching, merging, conflict resolution	DSA revision continues
7 Spring Boot	Spring Boot	DSA revision continues
8 Projects	Building 2-3 real projects	DSA revision + Git branching used naturally per feature
9 Docker/Linux/Cloud/Testing	Those topics	DSA revision + light system design reading
10 System Design +	Primary shifts back	Final polish on

Interviews	to DSA/system design/mock interviews	projects + resume
------------	--------------------------------------	-------------------

One-line version to remember: *Learn a Subject alone. Use a Tool daily without thinking about it. Never run two Subjects at once — but a Subject can run alongside a downgraded Maintenance habit once you’ve already been through it once.*

13. Weekly Rhythm (applies at every stage)

- **Mon-Fri:** classes/work + one Daily Loop session.
- **Saturday:** longer session — extra project/DSA time, catch up on weak spots.
- **Sunday:** revise the week, plan next week, no brand-new topic.

14. Exam / Busy Periods

- Normal day: 1 hr college + 1 hr Java/DSA + 45 min practice + 15 min review.
- Busy day: 45 min college + 45 min Java/DSA + 30 min practice.
- Exam month: 70-80% effort → academics, 20-30% → light maintenance (don’t fully stop).
- After exams: return to full routine immediately.

15. Monthly Checkpoint (ask yourself every month, at any stage)

- ☐ Can I explain this month’s concepts with no notes?
- ☐ Can I write related code without copying?
- ☐ Can I solve basic problems independently?
- ☐ What are my 3 weakest topics right now?
- ☐ Did I actually build something this month?
- ☐ Is college under control?

If you fail — don’t restart the whole roadmap. Spend a few extra days fixing the specific weak topic, then continue.

16. Hard Rules — Never Break These

1. Never study two languages/frameworks at once.
 2. Never start Spring Boot before finishing Java OOP.
 3. Never jump to hard DSA problems early — earn your way up from easy.
 4. Never watch tutorials for more than 30 min without coding something yourself.
 5. Never copy a GitHub project and call it yours.
 6. Never skip a stage checkpoint “just this once.”
 7. Never compare your Day 20 to someone else’s Day 500.
-

Start Right Now

Open VS Code → do the Setup in Section 0 → then do **Week 1, Day 1** in Section 3. Nothing else. Just that.